Diseño de Seguridad: Roles, Permisos y Auditoría (RBAC)

Proyecto: Sistema de Gestión y Migración Académica (SGMA)

Versión: 1.0

Fecha: Noviembre de 2026

Estado: Borrador, Fase 2 (tarjeta "Definir el esquema de roles (RBAC) y las tablas de Auditoría")

Base: SRS v1.2 (RF-USR-01 y 02, RNF-03 a 05, RNF-14) y esquema `gestion_academica.sql` v1.2

1. Resumen

El sistema tiene cuatro roles (Administrador, Preceptor, Docente y Directivo). Este documento define qué puede hacer cada uno sobre cada dato y función, cómo se hace cumplir en tres capas (aplicación, filtro por materia asignada y cuenta de base de datos con privilegios mínimos) y cómo la auditoría sabe qué persona hizo cada cambio. Al final quedan siete decisiones abiertas, porque la matriz de permisos agrega criterios que el SRS no fija.

2. Roles

Rol	Quién es	Alcance de los datos
ADMIN	Personal técnico / TI	Todo el sistema, pero con lectura sobre los datos académicos. Escribe datos académicos solo mediante la migración y la carga masiva.
PRECEPTOR	Personal de Bedelía	Todos los datos académicos de la institución.
DOCENTE	Personal docente	Solo las materias que tiene asignadas en `asignaciones_docentes`.
DIRECTIVO	Autoridades	Lectura global, incluida la auditoría.

El alumno no es usuario del sistema en esta versión (sección 8).

3. Matriz de permisos sobre los datos

C = crear, R = consultar, U = modificar, B = baja lógica (campo `activo` o `activa`). "Propias" significa solo las materias que el docente tiene asignadas en el ciclo correspondiente. Un guion indica que no tiene acceso.

Entidad	ADMIN	PRECEPTOR	DOCENTE	DIRECTIVO
Usuarios y roles	C R U B	—	—	—
Parámetros del sistema	R U	R	—	R
Alumnos	R	C R U B	R (solo alumnos de sus materias; legajo, apellido y nombre)	R
Matrículas	R	C R U	—	R
Carreras, materias y correlatividades	R	C R U B	R	R
Docentes y asignaciones	R	C R U B	R (su propio registro)	R
Ciclos lectivos y mesas de examen	R	C R U	R (propias)	R
Inscripciones a cursada y a examen	R	C R U	R (propias)	R
Notas de cursada	R	C R U	C R U (propias)	R
Actas de examen	R	C R U B	C R (propias)	R
Asistencias	R	C R U	C R U (propias)	R
Lotes de carga	R	C R (los propios)	—	R
Auditoría	R	—	—	R

Nadie escribe en `auditoria`: solo los triggers. Todos los usuarios pueden cambiar su propia contraseña.

4. Matriz de permisos sobre funciones

Función	ADMIN	PRECEPTOR	DOCENTE	DIRECTIVO	Requisito
Gestionar usuarios y roles	Sí	—	—	—	RF-USR-01
Cambiar parámetros (asistencia mínima)	Sí	—	—	—	RF-ASI-03
Ejecutar la migración inicial (ensayo y definitiva)	Sí	—	—	—	RF-MIG-05
Carga masiva por plantilla	Sí	Sí	—	—	RF-CMA-01 a 04
Buscar por DNI o legajo	Sí	Sí	Solo alumnos de sus materias	Sí	RF-REP-01
Certificado analítico en PDF	—	Sí	—	Sí	RF-REP-02
Exportar listados de alumnos y actas	—	Sí	Solo de sus materias	Sí	RF-REP-03
Reportes de asistencia	—	Sí	Solo de sus materias	Sí	RF-REP-04
Consultar la auditoría	Sí	—	—	Sí	RNF-05

5. Cómo se hacen cumplir los permisos

Los permisos se verifican en tres capas, de modo que un error en una no deje el sistema abierto.

1. **Aplicación:** cada pantalla y cada operación comprueba el rol del usuario contra las matrices de las secciones 3 y 4 antes de ejecutarse.
2. **Materias propias del docente:** para toda carga de un docente, la aplicación comprueba que la materia esté asignada. Si no lo está, rechaza con ERR_07 y registra el intento. La comprobación es esta:

```
SELECT 1
  FROM asignaciones_docentes ad
  JOIN usuarios u ON u.id_docente = ad.id_docente
 WHERE u.id_usuario = ? AND ad.id_materia = ? AND ad.id_ciclo = ?;
```

3. **Base de datos:** la aplicación se conecta con una cuenta de privilegios mínimos. No tiene DELETE ni permisos de estructura, y no puede escribir en `auditoria`. Así, aunque la aplicación tuviera un error, no podría borrar registros académicos ni alterar el historial de cambios.

```
-- La ejecuta la cuenta de administración, que es la misma que corre
gestion_academica.sql
CREATE USER 'sgma_app'@'localhost' IDENTIFIED BY '<clave definida fuera del
repositorio>';

GRANT SELECT, INSERT, UPDATE ON gestion_academica.alumnos TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.matriculas TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.docentes TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.usuarios TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.carreras TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.materias TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.correlatividades TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.ciclos_lectivos TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.asignaciones_docentes TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.inscripciones TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.notas_cursada TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.asistencias TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.mesas_examen TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.inscripciones_examen TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.lotes_carga TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.actas_examen TO
'sgma_app'@'localhost';
GRANT SELECT, INSERT, UPDATE ON gestion_academica.parametros TO
'sgma_app'@'localhost';
GRANT SELECT ON gestion_academica.roles TO 'sgma_app'@'localhost';
GRANT SELECT ON gestion_academica.auditoria TO 'sgma_app'@'localhost';
GRANT EXECUTE ON gestion_academica.* TO 'sgma_app'@'localhost';
```

Los triggers y los procedimientos se ejecutan con los privilegios de quien los creó (la cuenta de administración), por eso la cuenta `sgma_app` puede disparar la auditoría sin tener permiso de escritura sobre esa tabla. Este comportamiento está sin probar: hay que verificarlo cuando se ejecute el script.

6. Auditoría: cómo se sabe quién hizo cada cambio

Los triggers registran las altas, cambios y bajas de actas, notas de cursada y asistencias (RNF-05), pero solo conocen la cuenta de base de datos, que es siempre la misma. La persona se identifica con la variable de sesión `@app_user_id`, que la aplicación debe fijar en cada solicitud:

```
al iniciar cada solicitud autenticada:
    conexion = pool.obtener()
    conexion.ejecutar("SET @app_user_id = %s", [usuario.id_usuario])
    ... operaciones, dentro de una transacción ...
al terminar, también si hubo error:
    conexion.ejecutar("SET @app_user_id = NULL")
    pool.devolver(conexion)
```

El reinicio al final es obligatorio. Con un pool de conexiones, la variable queda en la conexión y, si no se limpia, la siguiente solicitud podría auditarse a nombre de otra persona. Si una persona edita la base directamente, sin pasar por la aplicación, el evento queda con `id_usuario` nulo y eso sirve como señal de que el cambio no vino de la aplicación.

Prueba mínima para la Fase 4, con un usuario ya creado:

```
SET @app_user_id = 1;
UPDATE actas_examen SET nota = 8 WHERE id_acta = 1;
SELECT id_usuario, tabla, operacion, campo, valor_anterior, valor_nuevo
  FROM auditoria ORDER BY id_evento DESC LIMIT 5;
```

Debe aparecer una fila con `id_usuario` = 1, el campo `nota` y los valores anterior y nuevo.

7. Ciclo de vida de un usuario

- **Alta:** la hace el Administrador. Un trigger impide crear un usuario con rol DOCENTE sin docente vinculado (RF-USR-02), y cada docente tiene como máximo un usuario.
- **Primer Administrador:** se crea a mano, con un `password_hash` real, porque el script no incluye claves.

- **Baja:** se pone `activo` en falso. Los usuarios no se eliminan, porque la auditoría y los lotes de carga los referencian.
- **Contraseñas:** se guardan como bcrypt o Argon2 (RNF-03). La aplicación nunca almacena ni registra la contraseña en texto plano.

8. Extensión futura: rol Alumno (RNF-14)

Incorporar al alumno no requiere rediseñar el esquema. Alcanza con agregar el valor ALUMNO a `roles`, una columna `id_alumno` única y nula en `usuarios` (análoga a `id_docente`), y un trigger equivalente al de los docentes. La regla de acceso sería "cada alumno ve solo su propio historial".

9. Decisiones abiertas

Nº	Tema	Propuesta	Por qué importa
1	Borrado lógico incompleto	Mantener sin DELETE para la aplicación. Si hace falta anular registros, agregar `activo` a las tablas que hoy no lo tienen (inscripciones, mesas, notas, asistencias).	El SRS (2.4) prohíbe la eliminación física, pero esas tablas no tienen campo de baja. Hoy una inscripción cargada por error no se puede quitar.
2	Docente y corrección de actas	El docente carga y consulta actas, pero solo el Preceptor las corrige.	El SRS dice que el docente "carga" actas y no menciona correcciones.
3	Datos del alumno visibles para el docente	Solo legajo, apellido y nombre, sin DNI ni e-mail.	Es lo mínimo que necesita para cargar notas.
4	Administrador con lectura sobre datos académicos	Lectura y no edición, salvo por la migración y la carga masiva.	El SRS no le da permisos de edición sobre datos académicos.
5	Política de sesión y contraseñas	Bloqueo tras 5 intentos fallidos e inactividad de 30 minutos (valores propuestos).	El SRS solo exige encriptar las credenciales.

Nº	Tema	Propuesta	Por qué importa
6	Auditoría de alumnos y usuarios	Ampliar los triggers a cambios en DNI, legajo y rol.	RNF-05 cubre solo calificaciones y asistencias, y hoy un cambio de DNI no deja rastro.
7	Verificación en un servidor real	Ejecutar el script, crear `sgma_app` y correr la prueba de la sección 6.	Los privilegios, el comportamiento de los triggers con esa cuenta y la variable de sesión no se probaron.